



Практическое задание: UDP-сервер и клиент для передачи текстовых сообщений

Цель

Создать пару приложений (сервер и клиент), которые демонстрируют ключевые особенности UDP: отсутствие установления соединения, независимую обработку каждого сообщения, возможность потери данных и работу с несколькими клиентами без поддержания состояния.

Задание

Часть 1: Сервер (UDP-приёмник)

1. Базовый сервер

- Создайте консольное приложение на C#
- Реализуйте сервер, который прослушивает указанный порт (например, 5000)
- При запуске сервер должен запрашивать порт у пользователя или использовать порт по умолчанию
- Сервер должен принимать датаграммы от любых клиентов

2. Обработка сообщений

- Каждое полученное сообщение должно выводиться в консоль вместе с:
 - IP-адресом и портом отправителя
 - Временем получения
 - Длиной сообщения в байтах
 - Текстом сообщения (если оно текстовое)
- Сервер должен продолжать работу до получения команды завершения (например, нажатия клавиши)

3. Статистика

- Реализуйте сбор статистики:
 - Количество полученных сообщений

- Общий объём полученных данных в байтах
- Список уникальных клиентов (по IP:порт)

Часть 2: Клиент (UDP-отправитель)

1. Базовый клиент

- Создайте отдельное консольное приложение
- Клиент должен запрашивать:
 - IP-адрес сервера (или использовать localhost по умолчанию)
 - Порт сервера
 - Сообщение для отправки

2. Отправка сообщений

- Реализуйте режимы работы:
 - Отправка одного сообщения
 - Отправка нескольких сообщений с указанным интервалом
 - Режим "эхо-тест": отправка сообщения и ожидание ответа от сервера
- Клиент должен отображать статус отправки (успешно/ошибка)

3. Обработка ответов

- В режиме "эхо-тест" сервер должен отвечать клиенту тем же сообщением
- Клиент должен замерять время между отправкой и получением ответа (RTT)

Часть 3: Демонстрация особенностей UDP

1. Моделирование потери пакетов

- Добавьте в сервер опцию случайного "игнорирования" сообщений (например, каждое 5-е сообщение не обрабатывается)
- Продемонстрируйте, что клиент не замечает потерю (если не реализована подтверждающая логика)

2. Работа с несколькими клиентами

- Запустите один сервер и несколько клиентов
- Покажите, что сервер обрабатывает сообщения от всех клиентов независимо

3. Порядок доставки

- Отправьте с одного клиента несколько сообщений с небольшими задержками
- Продемонстрируйте, что порядок получения может отличаться от порядка отправки

Подсказки по ключевым частям

Для сервера

- Используйте класс `UdpClient` для упрощения работы
- Для получения данных используйте метод `ReceiveAsync` для асинхронной работы
- Храните статистику в простых переменных-счётчиках
- Для вывода информации об отправителе используйте `EndPoint`

Для клиента

- Преобразуйте строки в байты с помощью `Encoding.UTF8.GetBytes()`
- Для отправки используйте `SendAsync`
- Реализуйте таймауты при ожидании ответа (например, 5 секунд)

Для эхо-функциональности

- Сервер должен отправлять ответ тому же endpoint, от которого получил сообщение
- Используйте перегрузку `SendAsync`, которая принимает данные и `EndPoint`

Что проверить перед отправкой (чек-лист)

Сервер

- ☐ Сервер запускается на указанном порту
- ☐ Выводит понятное сообщение при старте (адрес и порт)
- ☐ Принимает сообщения от клиентов
- ☐ Корректно отображает информацию об отправителе
- ☐ Обрабатывает несколько клиентов одновременно
- ☐ Собирает и отображает статистику
- ☐ Корректно завершается по команде пользователя

Клиент

- ☐ Позволяет ввести адрес сервера и порт

- ☐ Отправляет сообщения корректно
- ☐ Реализованы все три режима работы
- ☐ В режиме эхо-теста измеряется и выводится RTT
- ☐ Обрабатываются ошибки (недоступный сервер, таймауты)

Демонстрация

- ☐ Показана работа с несколькими клиентами
- ☐ Продемонстрирована возможность потери пакетов
- ☐ Показано, что порядок доставки не гарантирован

Советы по улучшению работы

1. Логирование

- Добавьте запись логов в файл (полученные сообщения, ошибки)
- Реализуйте разные уровни логирования (информация, предупреждения, ошибки)

2. Конфигурация

- Вынесите настройки (порт по умолчанию, таймауты) в конфигурационный файл
- Добавьте поддержку аргументов командной строки

3. Устойчивость

- Обрабатывайте исключения (SocketException, ObjectDisposedException)
- Добавьте возможность перезапуска сервера без перезапуска приложения

4. Производительность

- Используйте `Memory<T>` и `Span<T>` для работы с буферами
- Реализуйте пул буферов для уменьшения аллокаций памяти

5. Дополнительные функции

- Добавьте простой протокол с подтверждением доставки
- Реализуйте нумерацию сообщений для отслеживания потерь
- Добавьте поддержку команд (например, "статистика", "очистить")